

Тестабилни дизајн, NUnit, Moq и покривеност кода

Тестови као доказ пословног понашања и квалитета архитектонских граница.

Пројектовање тестабилног кода

Раздвајање одговорности, јасни уговори и убризгавање зависности омогућавају контролисано тестирање.

Раздвајање одговорности, јасни уговори и `dependency injection` омогућавају да се компоненте проверавају изоловано и детерминистички.

НАПОМЕНА

Тестови обезбеђују повратну информацију о квалитету архитектонских граница.

Тумачење резултата у Test Explorer-у

Неуспешан тест усмерава дијагностику, а укупан пролаз потврђује само извршене сценарије.

Test Explorer

ERS.Tests

- ✓ Reserve_WhenFree_ReturnsSuccess
- ✓ Reserve_WhenOverlap_ReturnsConflict
- ✓ Reserve_WhenInactive_ReturnsFailure
- ✗ Cancel_AfterDeadline_ReturnsFailure

3 теста су прошла, један захтева анализу.

Како се чита резултат

Црвени тест прво треба разумети, па тек онда мењати код.

Cancel_AfterDeadline_ReturnsFailure

Очекивано: CancellationWindowExpired

Добијено: Success

Тумачење

Порука показује да ли је проблем у правилу, имплементацији или очекивању теста. Код се мења тек после разумевања узрока.

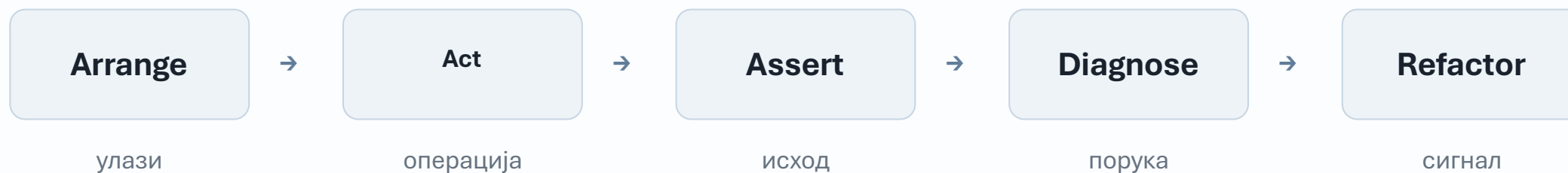
ПРОШЛО 3

ПАЛО 1

ТРАЈАЊЕ 184 ms

AAA структура теста

Један тест проверава једно јасно понашање.



НАПОМЕНА

Назив теста описује понашање које тест штити.

Негативни сценарио из критеријума прихватања

Тест проверава понашање система, а не приватне детаље имплементације.

```
1 [Test]
2 public void Reserve_WhenPeriodOverlaps_ReturnsConflict()
3 {
4     var repository = new Mock<IReservationRepository>();
5     repository.Setup(r => r.HasOverlap(It.IsAny<Guid>(), It.
6         IsAny<DateTime>(), It.IsAny<DateTime>()))
7         .Returns(true);
8
9     var result = service.Reserve(TestRequests.
10         ValidFutureReservation());
11
12     Assert.That(result.Success, Is.False);
13     Assert.That(result.Error, Is.EqualTo("OverlappingReservat
14         ion"));
15 }
```

НАПОМЕНЕ

Arrange припрема контролисаног сарадника.

Act извршава једну релевантну операцију.

Assert проверава посматрани исход.

Улога test double објеката

Test double се користи за изолацију конкретне спољне зависности.

Мали value object или доменска класа чије понашање је предмет теста најчешће треба да буде стварни објекат, не mock.

НАПОМЕНА

Mock објекти су оправдани на променљивим спољним границама: репозиторијум, HTTP клијент, време и случајност.

Критеријуми за употребу mock објекта

Mock објекат контролише променљиву спољну зависност чија имплементација није предмет теста.

Ситуација	Одлука
Repository или HTTP клијент	mock контролише повратну вредност
IClock или generator ID	mock обезбеђује детерминизам
Мали value object	користити стварни објекат
Доменска класа која се тестира	не mock-овати предмет теста

Moq: Setup и Verify

Setup контролише сарадника, а Verify проверава значајну интеракцију.

```
1 var repository = new Mock<IReservationRepository>();
2 repository.Setup(r => r.FindById(reservationId))
3             .Returns(existingReservation);
4
5 var result = service.Cancel(reservationId);
6
7 repository.Verify(r => r.Save(existingReservation), Times.
8                 Once);
```

НАПОМЕНЕ

Verify је оправдан када је интеракција део понашања.

Превише Verify провера прави крхак тест.

Не проверавати сваки интерни позив без потребе.

Извођење тест сценарија из правила

Тестови са највећом вредношћу покривају пословне ризике и граничне услове.

01

Успешан ток

валидни подаци и очекиван
резултат

02

Негативни ток

очекивано одбијање
захтева

03

Гранични случај

вредност која мења
понашање

План тестирања резервисања

План наводи сценарио, очекивани исход и потребне зависности.

Врста	Пример
Успешан сценарио	опрема је слободна, корисник овлашћен, термин валидан
Негативни сценарио	термин се преклапа са постојећом резервацијом
Негативни сценарио	опрема је неактивна
Гранични сценарио	нови термин почиње тачно када се претходни завршава
Инфраструктурни квар	repository пријављује грешку

Покретање тестова и анализа покривености

Покривеност кода показује извршене гране и служи као дијагностички сигнал.

```
1 dotnet test
2
3 # ако је collector подешен
4 dotnet test --collect:"XPlat Code Coverage"
```

НАПОМЕНЕ

Најпре дефинисати пословно важне сценарије.

Coverage користити као дијагностички сигнал.

Висок проценат не доказује одсуство грешака.

Ограничења показатеља покривености

Проценат покривености не замењује анализу пословног ризика и квалитета теста.

Питање није „колики је проценат”, већ који непокривени део носи највећи ризик и који тест би дао највише нових информација.

НАПОМЕНА

Извештај о покривености служи за идентификацију непроверених ризика.

Анализа извештаја о покривености

Извештај служи за идентификацију непроверених грана и ризика.

01

Условне гране

да ли су оба исхода проверена

02

Обрада грешака

да ли грешка не постаје лажни успех

03

Критично стање

новац, дозволе, резервације

04

Непокривен ризик

образложити или покрити тестом

Контролна тачка P3

До ове тачке функционално језгро мора бити ручно разумљиво и тестирано.

manual-core-baseline

стабилна верзија

После ове контролне тачке AI добија већу улогу, али систем већ има структуру и тестове који омогућавају независну проверу.

P3: manual-core-baseline

Контролна тачка раздваја ручно пројектовано језгро од почетка AI фазе.

- 01 Кључни use-case-ови имају NUnit тестове.
- 02 Моq се користи само за променљиве спољне зависности.
- 03 Coverage је прегледан и ризична грана је покривена или образложена.
- 04 Најмање један bug је прво репродукован тестом.
- 05 Стабилна верзија језгра означена је Git tag-ом manual-core-baseline.

Стабилност теста при рефакторисању

Тест треба да остане стабилан када се мења приватна имплементација без промене понашања.

ТЕСТИРАТИ

видљиви резултат

стабилан код неуспеха

значајну интеракцију

гранично понашање

НЕ ВЕЗИВАТИ СЕ ЗА

приватну методу

редослед непотребних позива

тренутну структуру петље

тривијалне setter-e

Тест за репродукцију грешке

Тест прво репродукује проблем, а затим потврђује исправку и спречава регресију.

01

Reproduce

тест показује тренутни bug

02

Fix

минимална измена
понашања

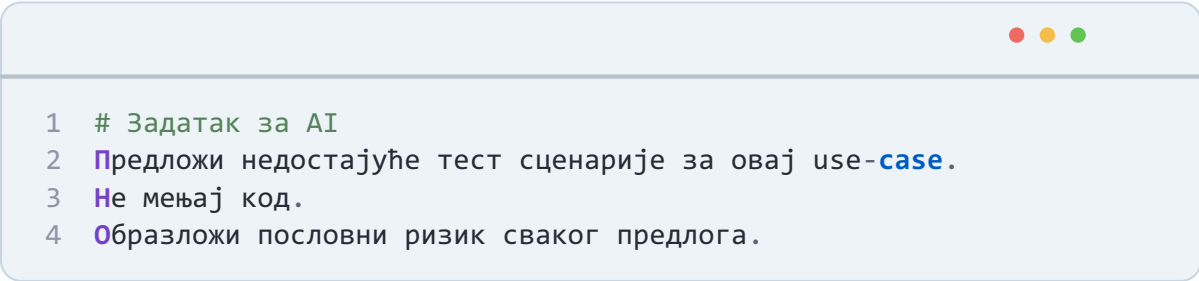
03

Protect

тест остаје као регресиона
заштита

Самостални задатак: AI предлог тест сценарија

AI може предложити сценарије, а тим бира релевантне тестове и образлаже избор.

- 
- A light blue rounded rectangle with a thin border and three colored dots (red, yellow, green) in the top right corner, resembling a window or a code editor header.
- 1 # Задатак за AI
 - 2 предложи недостајуће тест сценарије за овај use-case.
 - 3 Не мењај код.
 - 4 Образложи пословни ризик сваког предлога.

НАПОМЕНЕ

Први бод: нови смислен гранични тест.

Други бод: образложење прихватања или одбијања предлога.

Одговорност остаје код студента.

Резиме

Добре архитектонске границе омогућавају кратке и стабилне тестове усмерене на пословно понашање.

Када су границе система добре, тестови постају краћи, јаснији и ближи пословном понашању које студент треба да одбрани.

НАПОМЕНА

Ови резултати су основа за интеграцију и контролисан AI радни ток.

Припрема за интеграцију

Припремити стабилно функционално језгро и тестове за интеграцију модула.

- 01 Написати план тестирања за један use-case пре тест кода.
- 02 Додати успешне, негативне и граничне NUnit тестове.
- 03 Mock користити само на јасним спољним границама.
- 04 Покренути coverage и анализирати једну ризичну грану.
- 05 Означити стабилну верзију језгра када је спремна.